

FICHE DE TD 3/3 • DIFFICILE

Informatique • Tle C D E

Programmation en langage C

Module 3 : Algorithmique et programmation

Ce TD de synthèse évalue votre capacité à concevoir, structurer et analyser des programmes C complets mobilisant tableaux, structures, fichiers et algorithmes classiques.

EXERCICE 1 Gestion d'un stock de pièces détachées – Structures et fichiers

Une entreprise gère un stock de pièces détachées. Chaque pièce est caractérisée par : un code entier (4 chiffres), une désignation (chaîne de 30 caractères max), un prix unitaire (réel), et une quantité en stock (entier). On souhaite développer un module de gestion en langage C.

1. Définir un type `Piece` à l'aide d'une structure (`struct`) adaptée. Déclarer un tableau `stock` pouvant contenir au maximum 500 pièces et une variable `nbPieces` pour le nombre effectif.
2. Écrire la fonction `void saisirPiece(Piece *p)` qui lit les informations d'une pièce depuis l'entrée standard. Préciser les précautions à prendre pour la lecture de la désignation (espaces autorisés) après une lecture numérique.
3. Écrire la fonction `int rechercherCode(const Piece stock[], int n, int code)` qui retourne l'indice de la pièce de code `code` dans les `n` premières cases du tableau, ou `-1` si absent. Justifier le choix du qualificatif `const` pour le paramètre tableau.
4. Proposer un algorithme (en C ou pseudo-code structuré) pour la fonction `void approvisionner(Piece stock[], int *n, int code, int qte)` qui augmente la quantité de la pièce `code` de `qte`. Si le code n'existe pas et que le stock n'est pas plein, la pièce est créée (saisie des autres champs) et ajoutée. Gérer le cas stock plein.
5. On souhaite sauvegarder le stock dans un fichier binaire `stock.dat`. Écrire la fonction `int sauvegarder(const Piece stock[], int n)` qui retourne le nombre d'éléments effectivement écrits. Expliquer pourquoi l'écriture binaire (`fwrite`) est préférable à l'écriture formatée (`fprintf`) ici, et citer un risque majeur lié à la portabilité des fichiers binaires de structures.

EXERCICE 2 Analyse et correction d'un tri – Pointeurs et tableaux

Un élève a écrit le programme suivant pour trier un tableau d'entiers par ordre croissant selon l'algorithme du tri à bulles optimisé (arrêt si aucun échange).

```
#include <stdio.h>
#define TAILLE 10
```

```
void triBulles(int *t, int n) {
    int i, j, temp, echange;
    for (i = 0; i < n; i++) {
        echange = 0;
        for (j = 0; j < n - i; j++) {
            if (t[j] > t[j+1]) {
                temp = t[j];
                t[j] = t[j+1];
                t[j+1] = temp;
                echange = 1;
            }
        }
        if (echange == 0) break;
    }
}

int main() {
    int tab[TAILLE] = {5, 2, 8, 1, 9, 3, 7, 4, 6, 0};
    triBulles(tab, TAILLE);
    for (int k = 0; k <= TAILLE; k++) printf("%d ", tab[k]);
    return 0;
}
```

1. Identifier et corriger les trois erreurs majeures (logiques ou d'exécution) présentes dans ce code. Pour chacune, expliquer la conséquence exacte (accès mémoire, résultat incorrect, boucle infinie, etc.).
2. Réécrire la fonction `triBulles` en utilisant uniquement l'arithmétique des pointeurs (pas de crochets `[]`) pour accéder aux éléments du tableau. Le prototype doit rester `void triBulles(int *t, int n)`.
3. On souhaite désormais trier un tableau de structures `Piece` (définie à l'exercice 1) par prix croissant. Modifier le prototype et le corps de la fonction de tri pour qu'elle soit générique : `void triGenerique(void *base, size_t n, size_t taille, int (*compar)(const void *, const void *))`. Écrire la fonction de comparaison `int comparerPrix(const void *a, const void *b)` adaptée au type `Piece`.
4. Discuter de l'efficacité algorithmique du tri à bulles pour $n = 500$ (taille max du stock). Proposer un algorithme de tri standard de la bibliothèque C (`<stdlib.h>`) plus performant et donner sa complexité moyenne.

EXERCICE 3 Simulation d'une file d'attente – Allocation dynamique

On modélise une file d'attente d'entiers (FIFO) par une liste chaînée dynamique. La struc-

ture d'un nœud est :

```
typedef struct Noeud {
    int valeur;
    struct Noeud *suivant;
} Noeud;
```

La file est gérée par deux pointeurs `Noeud *tete, *queue`; (NULL si vide).

1. Écrire la fonction `int enfiler(Noeud **tete, Noeud **queue, int val)` qui insère `val` en fin de file. La fonction retourne 1 en cas de succès, 0 en cas d'échec d'allocation (`malloc`). Justifier l'utilisation du double pointeur `Noeud **`.
2. Écrire la fonction `int defiler(Noeud **tete, Noeud **queue, int *val)` qui retire l'élément en tête de file et stocke sa valeur dans `*val`. Retourner 1 si succès, 0 si file vide. Penser à libérer la mémoire (`free`) et à mettre à jour `queue` si la file devient vide.
3. Écrire une fonction `void viderFile(Noeud **tete, Noeud **queue)` qui libère toute la mémoire occupée par la file. Pourquoi ne pas simplement faire `free(*tete)` ?
4. Comparer cette implémentation par liste chaînée à une implémentation par tableau circulaire de taille fixe N pour une utilisation en système embarqué (mémoire limitée, temps réel). Citer deux avantages et deux inconvénients de chaque approche.

EXERCICE 4 Calcul numérique et analyse d'erreurs – `math.h` et types

On souhaite calculer une approximation de l'intégrale $I = \int_0^1 e^{-x^2} dx$ par la méthode des rectangles (point milieu) avec n subdivisions. La formule est $I_n = \frac{1}{n} \sum_{k=0}^{n-1} e^{-(x_k)^2}$ avec $x_k = \frac{k+0.5}{n}$.

1. Écrire une fonction `double integraleRectangles(int n)` qui calcule I_n . Inclure `<math.h>` et utiliser `exp()` et `pow()` ou la multiplication. Déclarer les variables de boucle et d'accumulation avec les types les plus appropriés (`size_t`, `double`, etc.) et justifier le choix de `double` plutôt que `float` pour l'accumulateur.
2. Écrire un `main` complet qui : lit un entier strictement positif n (vérification), appelle la fonction, affiche le résultat avec 10 décimales (`%.10f`), et affiche la différence absolue avec la valeur de référence $I_{ref} \approx 0.7468241328$ (constante `const double`).
3. Analyser l'erreur commise. Si l'on double n , comment l'erreur évolue-t-elle théoriquement pour cette méthode ? Écrire une boucle dans `main` qui affiche I_n et l'erreur pour $n = 10, 100, 1000, 10000, 100000$. Quel phénomène numérique observe-t-on pour les très grandes valeurs de n (somme de nombreux termes très petits) ? Citer le nom de ce phénomène.
4. Proposer une modification de la boucle de sommation (ordre des termes, compensation type Kahan) pour atténuer ce phénomène. Expliquer le principe en deux lignes.